Oracle Database 19c RAC 과정

인스턴스와 클러스터 DB 관리

구성 확인, 기동·정지, 파라미터, PDB

제9장

학습 목표

- srvctl config database 출력을 읽는다
- 인스턴스별 스레드·UNDO 배정을 확인한다
- PDB를 두 인스턴스에서 열고 상태를 저장한다
- 종료 방식에 따른 Clusterware 반응을 설명한다
- 파라미터 세 분류와 sid= 지정의 위험을 안다
- RESET으로 SPFILE을 기준선으로 되돌린다

Agenda

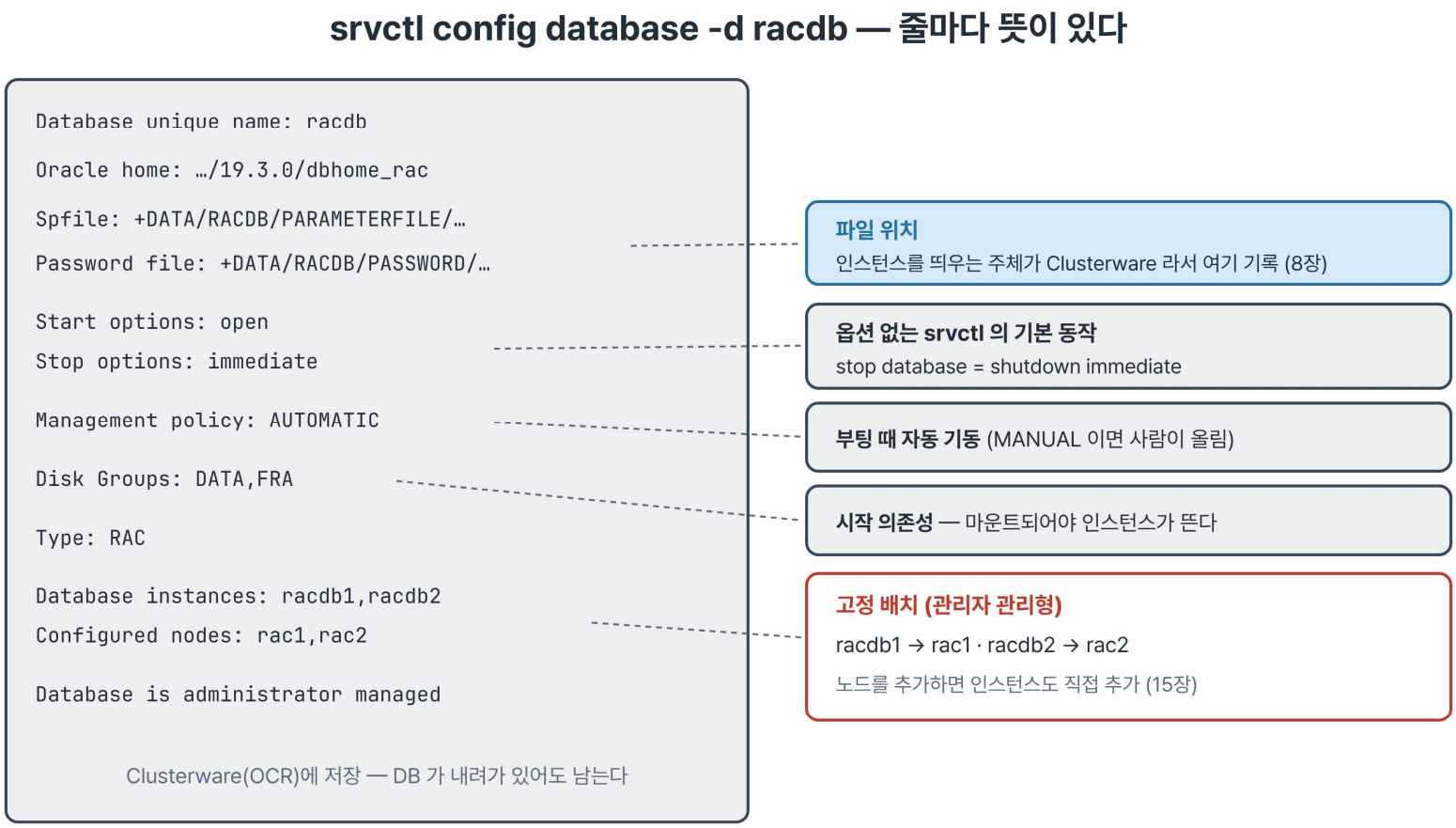
- 9.2 생성된 RAC DB의 구성 확인
- 9.3 srvctl · crsctl · SQL*Plus
- 9.4 SPFILE의 sid.parameter 표기
- 9.5 같아야 할 것을 한쪽만 바꾸면
- 9.6 PDB를 노드별로 열고 닫기
- 9.7 운영 지침

9.2

생성된 RAC DB의 구성 확인

- srvctl config database
- 인스턴스·스레드·UNDO
- PDB OPEN과 SAVE STATE

srvctl config database 읽기



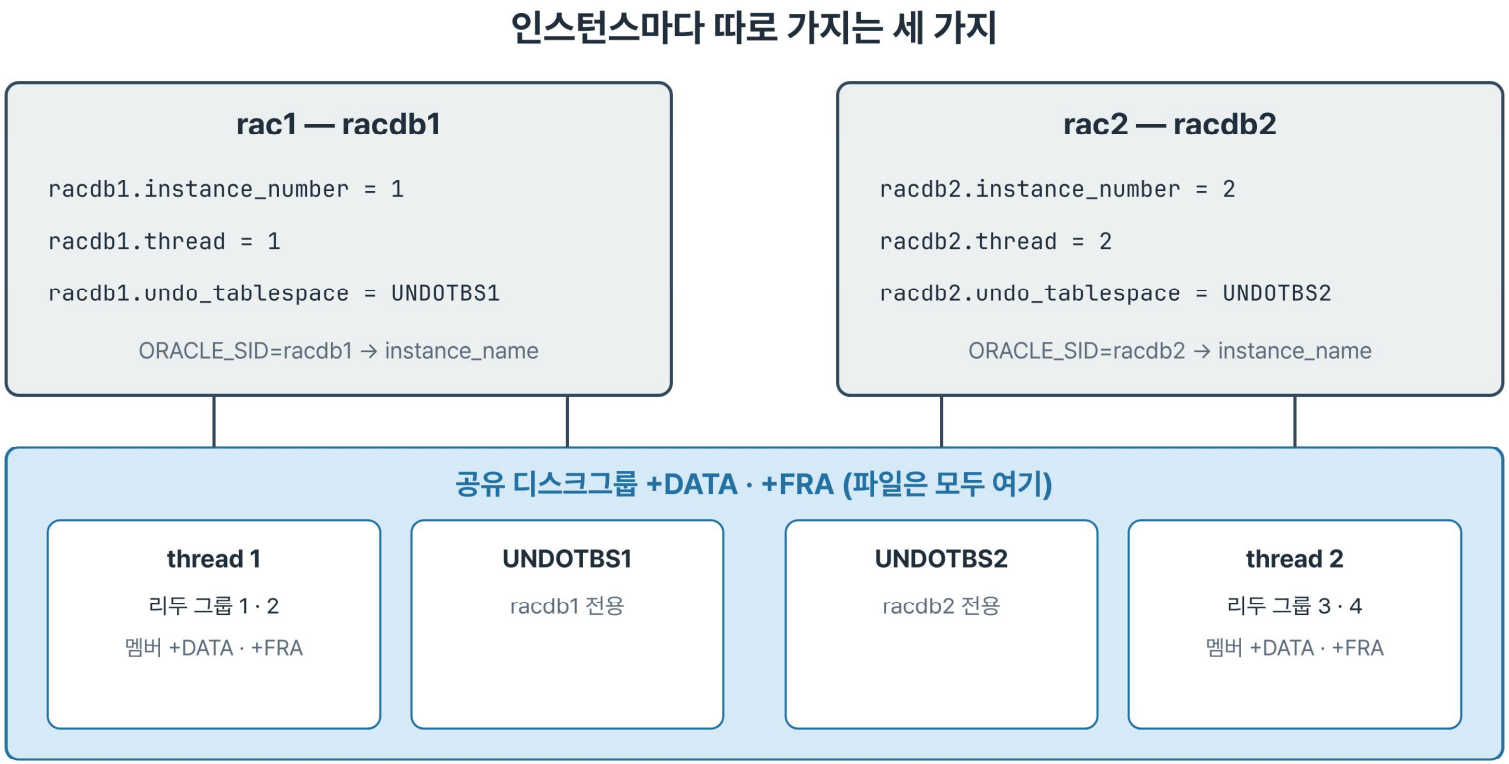
Clusterware에 저장된 racdb의 정의

출력에서 읽을 줄

기본 동작과 배치

- Start options open / Stop options immediate
- Management policy AUTOMATIC: 부팅 때 자동 기동
- Disk Groups DATA,FRA: 시작 의존성
- racdb1@rac1, racdb2@rac2, administrator managed

인스턴스마다 따로 가지는 것



셋 다 반드시 달라야 한다 — 같으면 리두·UNDO 를 두고 두 인스턴스가 충돌

파일이 공유돼 있어 한쪽이 죽으면 다른 쪽이 그 스레드와 UNDO 로 인스턴스 복구

번호, 리두 스레드, UNDO

스레드와 UNDO 확인

조회 네 개로 RAC임을 증명한다

- GV$INSTANCE: 서로 다른 노드, THREAD# 1·2
- V$LOG: 스레드마다 그룹 2개, CURRENT 하나씩
- UNDO 테이블스페이스 2개
- GV$PARAMETER: undo_tablespace가 인스턴스마다 다름

PDB OPEN과 SAVE STATE

DBCA는 저장하지 않는다

- 생성 직후 DBA_PDB_SAVED_STATES 0행
- 재기동하면 PDB가 두 노드 모두 MOUNTED
- OPEN INSTANCES=ALL + SAVE STATE INSTANCES=ALL
- 저장 상태도 인스턴스마다 한 행

9.3

srvctl · crsctl · SQL*Plus

- start와 stop의 옵션
- TARGET과 STATE
- 정상 종료와 비정상 종료

start와 stop의 옵션

SQL*Plus 인자에 RAC 옵션이 더해진다

- -startoption / -stopoption: startup·shutdown 인자
- -start/-stopconcurrency: 동시 처리 인스턴스 수
- -drain_timeout: 서비스 세션이 빠질 시간
- -eval로 영향 확인, -force는 stop에만

TARGET과 STATE

무엇을 유지하려 하는가, 지금은 어떤가

- srvctl stop instance racdb2 → 데이터베이스는 계속 열림
- crsctl stat res: 2번 OFFLINE OFFLINE
- srvctl stop은 TARGET도 OFFLINE으로
- 정지 약 50초, 기동 약 40초

종료 방식과 Clusterware의 반응

Clusterware 는 도구가 아니라 종료 방식을 본다

종료 방식	TARGET	STATE	Clusterware
srvctl stop instance 정상 종료	OFFLINE	OFFLINE	일치 → 그대로 둔다 재부팅해도 안 올라옴
SQL*Plus shutdown immediate 정상 종료 — 에이전트가 감지	OFFLINE	OFFLINE	일치 → 그대로 둔다 100초 관찰, pmon 0개
kill -9 pmon 비정상 종료	ONLINE	OFFLINE → STARTING (20초) → ONLINE (61초)	**어긋남 → 다시 올린다** RESTART_ATTEMPTS=2 UPTIME_THRESHOLD=1h

- TARGET = Clusterware 가 유지하려는 상태, STATE = 지금 실제 상태
- 그래도 srvctl: 의존 자원(서비스) 처리 · 어느 노드에서나 실행 · -drain_timeout
- srvctl 로 내린 것은 srvctl 로 올려야 TARGET 이 ONLINE 으로 돌아온다

정상 종료면 그대로, 비정상 종료면 다시 올린다

재시작 규칙

몇 번까지, 언제 잊는가

- RESTART_ATTEMPTS=2: 두 번까지 다시 올림
- 모두 실패하면 OFFLINE으로 둠
- UPTIME_THRESHOLD=1h: 1시간 정상이면 이력 초기화
- crsctl stat res ora.racdb.db -p로 확인

그래도 srvctl을 쓰는 이유

정지와 기동은 같은 도구로

- 의존 자원(서비스)을 순서대로 처리
- 어느 노드에서든 실행 가능
- -stopconcurrency, -drain_timeout
- srvctl로 내린 인스턴스는 재부팅해도 안 올라옴

9.4

SPFILE의 sid.parameter 표기

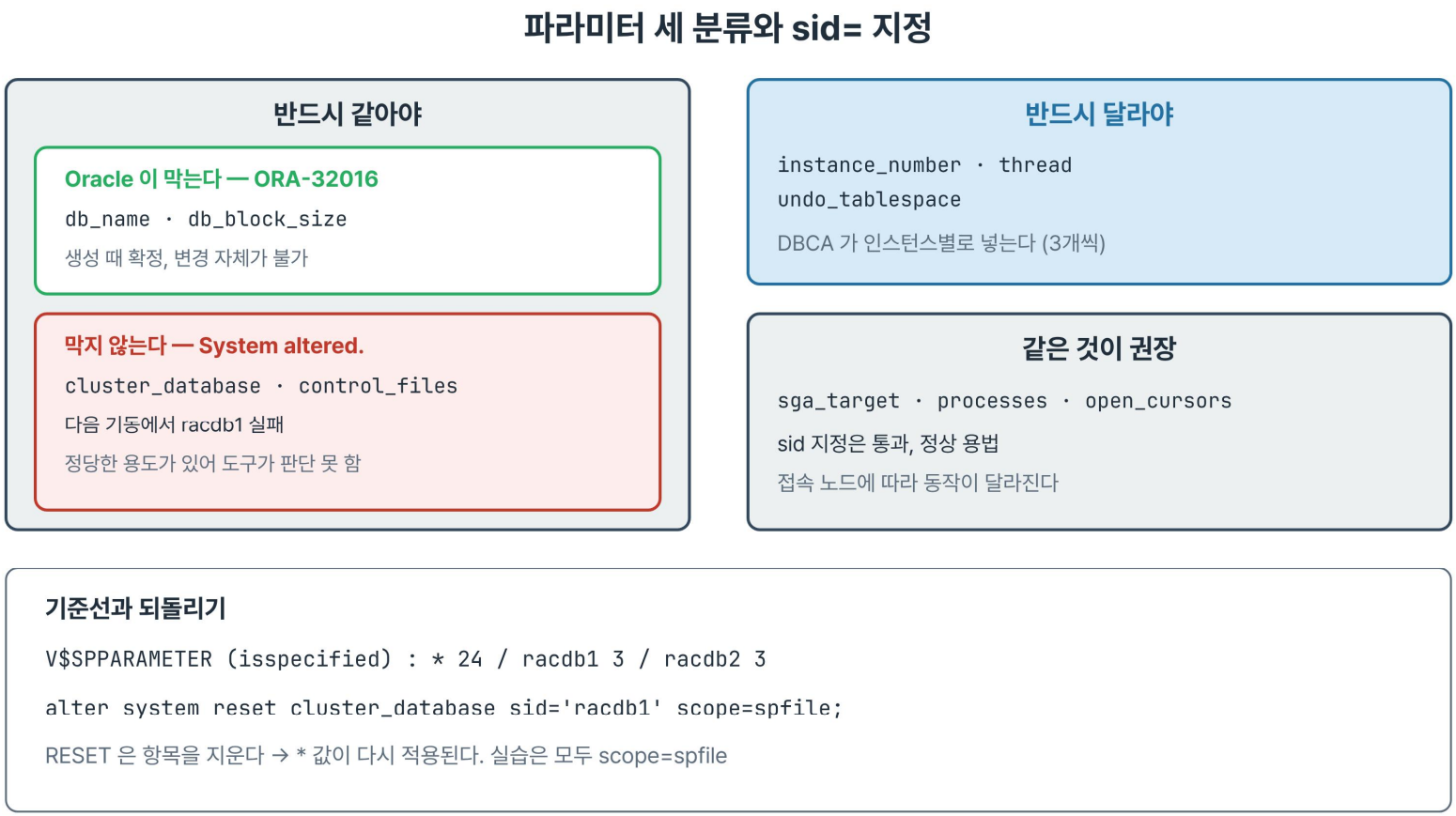
- * 와 인스턴스 이름
- 기준선 24 / 3 / 3
- 세 분류

* 와 인스턴스 이름

인스턴스별 값이 우선한다

- *.open_cursors=300: 모든 인스턴스
- racdb1.thread=1: 그 인스턴스만
- 공통 24, racdb1 3, racdb2 3
- 인스턴스별 셋: instance_number, thread, undo_tablespace

파라미터 세 분류



같아야 하는 것 중 일부만 Oracle이 막는다

9.5

같아야 할 것을 한쪽만 바꾸면

- ORA-32016으로 막히는 것
- 경고 없이 통과하는 것
- RESET으로 되돌리기

막는 경우

생성 때 확정된 값

- db_block_size=16384 sid='racdb1' scope=spfile
- db_name='xxx' sid='racdb1' scope=spfile
- ORA-32017 / ORA-32016
- 인스턴스별 지정 이전에 변경 자체가 불가

막지 않는 경우

명령은 맞고 상황이 틀렸다

- cluster_database=false sid='racdb1' → 통과
- control_files='+DATA/x' sid='racdb1' → 통과
- 다음 기동: MOUNT 실패, 고쳐도 배타 열기 실패
- 정당한 용도가 있어 도구는 막지 못함

RESET으로 되돌리기

값을 다시 쓰지 않고 항목을 지운다

- alter system reset cluster_database sid='racdb1' scope=spfile
- 항목이 사라지면 * 값이 다시 적용
- 24 / 3 / 3 복귀로 확인
- open_cursors 같은 권장 분류도 실습 뒤 RESET

scope 고르기

재기동 직전이 마지막 확인 기회

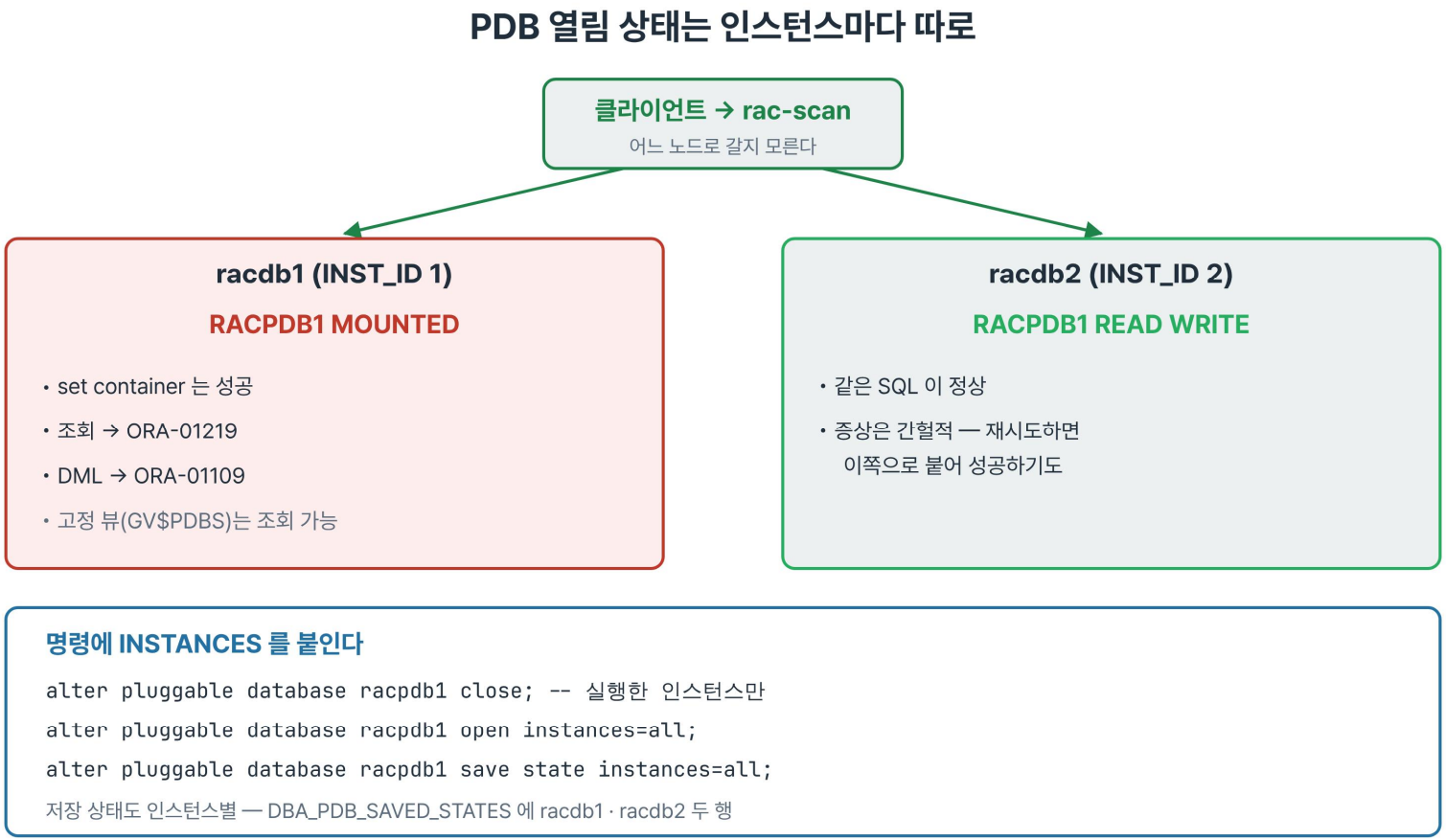
- memory: 지금 인스턴스만, 재기동하면 사라짐
- spfile: 다음 기동에 반영
- both: 기본값, 동적 파라미터에
- 실습은 모두 scope=spfile

9.6

PDB를 노드별로 열고 닫기

- 열림 상태는 인스턴스별
- ORA-01219와 ORA-01109
- OPEN INSTANCES=ALL

PDB 열림 상태는 인스턴스별



한 노드에서만 닫히면 그 노드 세션만 실패

간헐적 실패로 나타난다

SCAN으로 붙으면 노드를 모른다

- 한 노드에서만 닫힌 PDB
- 그 노드로 붙을 때만 실패, 재시도하면 성공하기도
- GV$PDBS의 INST_ID별 OPEN_MODE로 즉시 확인
- 의도적으로 한 노드만 열 때는 기록을 남김

9.7

운영 지침

- 도구 선택
- 일상 점검

무엇으로 내리고 올릴 것인가

SQL*Plus는 단일 인스턴스 모드 작업에만

- 인스턴스·DB 정지: srvctl stop
- 빠듯한 환경: -stopconcurrency 1
- 클러스터 전체: crsctl stop crs (root)
- 정지와 기동은 같은 도구로 짝지어

일상 점검

두 노드가 같은가를 묻는다

- 매일: srvctl status database -v, GV$PDBS
- 주간: SPFILE 24 / 3 / 3, TARGET·STATE 불일치
- 변경 후: GV$PARAMETER 두 노드 대조
- sid= 붙이기 전 세 분류 확인

요약

- srvctl config database는 Clusterware에 저장된 DB 정의
- 인스턴스마다 번호·리두 스레드·UNDO를 따로 가진다
- PDB는 OPEN과 SAVE STATE 모두 INSTANCES=ALL
- Clusterware는 도구가 아니라 종료 방식으로 판단
- cluster_database·control_files의 sid 지정은 통과한다
- RESET으로 항목을 지우고 24 / 3 / 3 확인

퀴즈

- 1. srvctl stop database의 기본 종료 방식은? 어디서 보나?
- 2. 생성 직후 확인할 항목 다섯 가지는?
- 3. 재기동 후 PDB가 MOUNTED로 남는 원인과 예방은?
- 4. shutdown immediate와 kill pmon의 반응 차이는?
- 5. cluster_database sid 지정이 통과하는 이유와 위험은?
- 6. 되돌릴 때 RESET을 쓰는 이유는?

실습 개요

- 11-05 · 11-06 생성 결과 검증, PDB 두 노드 (racdb로 실행)
- 12-01 ★ srvctl 기동·정지 / 12-03 ★ 정상·비정상 종료
- 12-04 SPFILE 표기 / 12-05 ★ 파라미터 세 분류
- 12-06 오류 재현과 RESET (24 / 3 / 3 복귀 필수)
- 파일: RAC/11장, RAC/12장 의 *_실습_NN_*.txt